

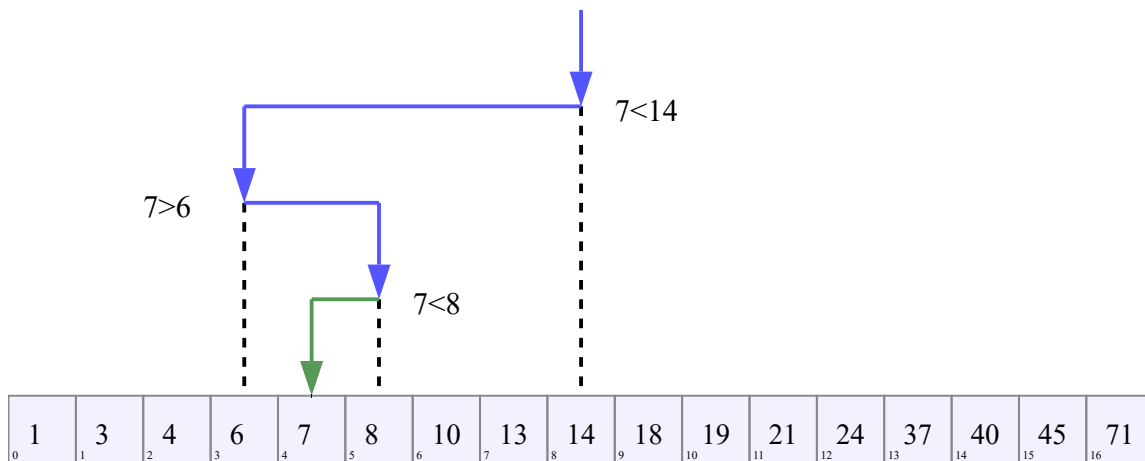
Original

Binary search

Binary search is a method that allows for quicker search of something by splitting the search interval into two. Its most common application is searching values in sorted arrays, however the splitting idea is crucial in many other typical tasks.

Search in sorted arrays

The most typical problem that leads to the binary search is as follows. You're given a sorted array $A_0 \leq A_1 \leq \dots \leq A_{n-1}$, check if k is present within the sequence. The simplest solution would be to check every element one by one and compare it with k (a so-called linear search). This approach works in $O(n)$, but doesn't utilize the fact that the array is sorted.



Binary search of the value 7 in an array.

The image by AlwaysAngry is distributed under CC BY-SA 4.0 license.

Now assume that we know two indices $L < R$ such that $A_L \leq k \leq A_R$. Because the array is sorted, we can deduce that k either occurs among $A_L, A_{L+1}, \dots, A_R$ or doesn't occur in the array at all. If we pick an arbitrary index M such that $L < M < R$ and check whether k is less or greater than A_M . We have two possible cases:

1. $A_L \leq k \leq A_M$. In this case, we reduce the problem from $[L, R]$ to $[L, M]$;
2. $A_M \leq k \leq A_R$. In this case, we reduce the problem from $[L, R]$ to $[M, R]$.

When it is impossible to pick M , that is, when $R = L + 1$, we directly compare k with A_L and A_R . Otherwise we would want to pick M in such manner that it reduces the active segment to a single element as quickly as possible in the worst case.

Since in the worst case we will always reduce to larger segment of $[L, M]$ and $[M, R]$. Thus, in the worst case scenario the reduction would be from $R - L$ to $\max(M - L, R - M)$. To minimize this value, we should pick $M \approx \frac{L+R}{2}$, then

$$M - L \approx \frac{R - L}{2} \approx R - M.$$

In other words, from the worst-case scenario perspective it is optimal to always pick M in the middle of $[L, R]$ and split it in half. Thus, the active segment halves on each step until it becomes of size 1. So, if the process needs h steps, in the end it reduces the difference between R and L from $R - L$ to $\frac{R-L}{2^h} \approx 1$, giving us the equation $2^h \approx R - L$.

Taking $\log_2$ on both sides, we get $h \approx \log_2(R - L) \in O(\log n)$.

Logarithmic number of steps is drastically better than that of linear search. For example, for $n \approx 2^{20} \approx 10^6$ you'd need to make approximately a million operations for linear search, but only around 20 operations with the binary search.

Lower bound and upper bound

It is often convenient to find the position of the first element that is greater or equal than k (called the lower bound of k in the array) or the position of the first element that is greater than k (called the upper bound of k) rather than the exact position of the element.

Together, lower and upper bounds produce a possibly empty half-interval of the array elements that are equal to k . To check whether k is present in the array it's enough to find its lower bound and check if the corresponding element equates to k .

Implementation

The explanation above provides a rough description of the algorithm. For the implementation details, we'd need to be more precise.

We will maintain a pair $L < R$ such that $A_L \leq k < A_R$. Meaning that the active search interval is $[L, R)$. We use half-interval here instead of a segment $[L, R]$ as it turns out to require less corner case work.

When $R = L + 1$, we can deduce from definitions above that R is the upper bound of k . It is convenient to initialize R with past-the-end index, that is $R = n$ and L with before-the-beginning index, that is $L = -1$. It is fine as long as we never evaluate A_L and A_R in our algorithm directly, formally treating it as $A_L = -\infty$ and $A_R = +\infty$.

Finally, to be specific about the value of M we pick, we will stick with $M = \lfloor \frac{L+R}{2} \rfloor$.

Then the implementation could look like this:

```
... // a sorted array is stored as a[0], a[1], ..., a[n-1]
int l = -1, r = n;
while (r - l > 1) {
    int m = (l + r) / 2;
    if (k < a[m]) {
        r = m; // a[l] <= k < a[m] <= a[r]
    } else {
        l = m; // a[l] <= a[m] <= k < a[r]
    }
}
```

During the execution of the algorithm, we never evaluate neither A_L nor A_R , as $L < M < R$. In the end, L will be the index of the last element that is not greater than k (or -1 if there is no such element) and R will be

the index of the first element larger than k (or n if there is no such element).

Note. Calculating m as $m = (r + 1) / 2$ can lead to overflow if l and r are two positive integers, and this error lived about 9 years in JDK as described in the [blogpost](#). Some alternative approaches include e.g. writing $m = 1 + (r - l) / 2$ which always works for positive integer l and r , but might still overflow if l is a negative number. If you use C++20, it offers an alternative solution in the form of `m = std::midpoint(l, r)` which always works correctly.

Search on arbitrary predicate

Let $f : \{0, 1, \dots, n-1\} \rightarrow \{0, 1\}$ be a boolean function defined on $0, 1, \dots, n-1$ such that it is monotonously increasing, that is

$$f(0) \leq f(1) \leq \dots \leq f(n-1).$$

The binary search, the way it is described above, finds the partition of the array by the predicate $f(M)$, holding the boolean value of $k < A_M$ expression. It is possible to use arbitrary monotonous predicate instead of $k < A_M$. It is particularly useful when the computation of $f(k)$ requires too much time to actually compute it for every possible value. In other words, binary search finds the unique index L such that $f(L) = 0$ and $f(R) = f(L+1) = 1$ if such a *transition point* exists, or gives us $L = n-1$ if $f(0) = \dots = f(n-1) = 0$ or $L = -1$ if $f(0) = \dots = f(n-1) = 1$.

Proof of correctness supposing a transition point exists, that is $f(0) = 0$ and $f(n-1) = 1$: The implementation maintains the *loop invariant* $f(l) = 0, f(r) = 1$. When $r - l > 1$, the choice of m means $r - l$ will always decrease. The loop terminates when $r - l = 1$, giving us our desired transition point.

```
... // f(i) is a boolean function such that f(0) <= ... <= f(n-1)
int l = -1, r = n;
while (r - l > 1) {
    int m = (l + r) / 2;
    if (f(m)) {
        r = m; // 0 = f(l) < f(m) = 1
    } else {
        l = m; // 0 = f(m) < f(r) = 1
    }
}
```

Binary search on the answer

Such situation often occurs when we're asked to compute some value, but we're only capable of checking whether this value is at least i . For example, you're given an array $a_1, \dots, a_n$ and you're asked to find the maximum floored average sum

$$\left\lfloor \frac{a_l + a_{l+1} + \dots + a_r}{r - l + 1} \right\rfloor$$

among all possible pairs of l, r such that $r - l \geq x$. One of simple ways to solve this problem is to check whether the answer is at least λ , that is if there is a pair l, r such that the following is true:

$$\frac{a_l + a_{l+1} + \dots + a_r}{r - l + 1} \geq \lambda.$$

Equivalently, it rewrites as

$$(a_l - \lambda) + (a_{l+1} - \lambda) + \dots + (a_r - \lambda) \geq 0,$$

so now we need to check whether there is a subarray of a new array $a_i - \lambda$ of length at least $x + 1$ with non-negative sum, which is doable with some prefix sums.

Continuous search

Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a real-valued function that is continuous on a segment $[L, R]$.

Without loss of generality assume that $f(L) \leq f(R)$. From [intermediate value theorem](#) it follows that for any $y \in [f(L), f(R)]$ there is $x \in [L, R]$ such that $f(x) = y$. Note that, unlike previous paragraphs, the function is *not* required to be monotonous.

The value x could be approximated up to $\pm\delta$ in $O\left(\log \frac{R-L}{\delta}\right)$ time for any specific value of δ . The idea is essentially the same, if we take $M \in (L, R)$ then we would be able to reduce the search interval to either $[L, M]$ or $[M, R]$ depending on whether $f(M)$ is larger than y . One common example here would be finding roots of odd-degree polynomials.

For example, let $f(x) = x^3 + ax^2 + bx + c$. Then $f(L) \rightarrow -\infty$ and $f(R) \rightarrow +\infty$ with $L \rightarrow -\infty$ and $R \rightarrow +\infty$. Which means that it is always possible to find sufficiently small L and sufficiently large R such that $f(L) < 0$ and $f(R) > 0$. Then, it is possible to find with binary search arbitrarily small interval containing x such that $f(x) = 0$.

Search with powers of 2

Another noteworthy way to do binary search is, instead of maintaining an active segment, to maintain the current pointer i and the current power k . The pointer starts at $i = L$ and then on each iteration one tests the predicate at point $i + 2^k$. If the predicate is still 0, the pointer is advanced from i to $i + 2^k$, otherwise it stays the same, then the power k is decreased by 1.

This paradigm is widely used in tasks around trees, such as finding lowest common ancestor of two vertices or finding an ancestor of a specific vertex that has a certain height. It could also be adapted to e.g. find the k -th non-zero element in a Fenwick tree.

Parallel Binary Search

- Note: Section Added my me.

When we are faced with multiple queries that can each be solved with a binary search, it is sometimes too slow to solve them one by one. Parallel Binary Search is a technique that allows us to solve all of these queries simultaneously, often leading to a significant performance improvement. The main idea is to perform the binary search for all queries at the same time (in parallel), step by step. This is particularly effective when the check function for the binary search is costly and can be optimized by processing queries in batches.

Consider a scenario where we have Q queries, and for each query q , we need to find the smallest value x that satisfies a certain condition $P(q, x)$. If $P(q, x)$ is monotonic on x , we can use binary search for each query. This would result in a total complexity of $O(Q \cdot \log(\text{range}) \cdot T_{\text{check}})$, where T_{check} is the time to evaluate $P(q, x)$.

Parallel binary search optimizes this by changing the order of operations. Instead of processing each query independently, we process all queries simultaneously, step by step. In each step of the binary search, we compute the middle points m_i for all queries and group the queries by their middle point. This is particularly powerful if the check function $P(q, x)$ has a structure that allows for efficient batching or updates.

Specifically, the major performance gain comes from two scenarios: 1. **Batching expensive checks:** If multiple queries need to check the same value m in a given step, we can perform the expensive part of the check only once and reuse the result. 2. **Efficiently updatable checks:** Often, the check for a value m (e.g., "process the first m events") can be performed much faster if we have already computed the state for $m - 1$. By processing the check values m in increasing order, we can update the state from one check to the next, instead of recomputing from scratch each time. This is a very common pattern in problems involving time or a sequence of updates.

This "offline" processing of queries, where we collect all queries and answer them together in a way that is convenient for our data structures, is the core idea behind parallel binary search.

Implementation

Imagine that we want to answer Z queries about the index of the largest value less than or equal to some X_i (for $i = 1, 2, \dots, Z$) in a sorted 0-indexed array A . Naturally, each query can be answered using binary search.

Specifically, let us consider the following array $A = [1, 3, 5, 7, 9, 9, 13, 15]$ with queries: $X = [8, 11, 4, 5]$. We can use binary search for each query sequentially.

Query	$X_1 = 8$	$X_2 = 11$	$X_3 = 4$	$X_4 = 5$
Step 1	Answer in $[0, 8)$ Check A_4 $X_1 < A_4 = 9$	Answer in $[0, 8)$ Check A_4 $X_2 \geq A_4 = 9$	Answer in $[0, 8)$ Check A_4 $X_3 < A_4 = 9$	Answer in $[0, 8)$ Check A_4 $X_4 < A_4 = 9$
Step 2	Answer in $[0, 4)$ Check A_2 $X_1 \geq A_2 = 5$	Answer in $[4, 8)$ Check A_6 $X_2 < A_6 = 13$	Answer in $[0, 4)$ Check A_2 $X_3 < A_2 = 5$	Answer in $[0, 4)$ Check A_2 $X_4 \geq A_2 = 5$
Step 3	Answer in $[2, 4)$ Check A_3 $X_1 \geq A_3 = 7$	Answer in $[4, 6)$ Check A_5 $X_2 \geq A_5 = 9$	Answer in $[0, 2)$ Check A_1 $X_3 \geq A_1 = 3$	Answer in $[2, 4)$ Check A_3 $X_4 < A_3 = 7$
Step 4	Answer in $[3, 4)$ $index = 3$	Answer in $[5, 6)$ $index = 5$	Answer in $[1, 2)$ $index = 1$	Answer in $[2, 3)$ $index = 2$

We generally process this table by columns (queries), but notice that in each row we often repeat access to certain values of the array. To limit access to these values, we can process the table by rows (steps). This does not make huge difference in our small example problem (as we can access all elements in $O(1)$), but in more complex problems, where computing these values is more complicated, this might be essential to solve these problems efficiently. Moreover, note that we can arbitrarily choose the order in which we answer questions in a single row. Let us look at the code implementing this approach.

```

// Computes the index of the largest value in a sorted array A less than or equal to X_i for
// all i.
vector<int> parallel_binary_search(vector<int>& A, vector<int>& X) {
    int N = A.size();
    int Z = X.size();
    vector<int> l(Z, -1), r(Z, N);

    for (int step = 1; step <= ceil(log2(N)); ++step) {
        // A vector of vectors to store indices of queries for each middle point.
        vector<vector<int>> m_to_queries(N);

        // Group queries by their middle point.
        for (int i = 0; i < Z; ++i) {
            if (l[i] < r[i] - 1) {
                int m = l[i] + (r[i] - l[i]) / 2;
                m_to_queries[m].push_back(i);
            }
        }

        // Process each group of queries.
        for (int m = 0; m < N; ++m) {
            if (m_to_queries[m].empty()) {
                continue;
            }
            for (int query : m_to_queries[m]) {
                if (X[query] < A[m]) {
                    r[query] = m;
                } else {
                    l[query] = m;
                }
            }
        }
    }
    return l;
}

```

Example Problem: Meteors

A pretty well known problem using this method is called "Meteors", and is listed in the practice problems. We are given N countries, and for each country, a target number of meteors to collect. We are also given a sequence of K meteor showers, each affecting a range of countries. The goal is to find, for each country, the earliest time (i.e., which meteor shower) they reach their target.

For a single country, we could binary search for the answer from 1 to K . The check for a given time t would involve summing up the meteors from the first t showers for that country. A naive check takes $O(t)$ time, leading to an overall complexity of $O(K \log K)$ for one country, and $O(N \cdot K \log K)$ for all, which is too slow.

With parallel binary search, we search for the answer for all N countries at once. In each of the $O(\log K)$ steps, we have a set of check values t_i for the countries. We can process these t_i in increasing order. To perform the check for time t , we can use a data structure like a Fenwick tree or a segment tree to maintain the meteor counts for all countries. When moving from checking time t_i to t_{i+1} , we only need to add the effects of showers from $t_i + 1$ to t_{i+1} to our data structure. This "update" approach is much faster than recomputing from scratch. The total complexity becomes $O((N + K) \log N \log K)$.

Translated

From: e-maxx.ru

Ternary Search

We are given a function $f(x)$ which is unimodal on an interval $[l, r]$. By unimodal function, we mean one of two behaviors of the function:

1. The function strictly increases first, reaches a maximum (at a single point or over an interval), and then strictly decreases.
2. The function strictly decreases first, reaches a minimum, and then strictly increases.

In this article, we will assume the first scenario. The second scenario is completely symmetrical to the first.

The task is to find the maximum of function $f(x)$ on the interval $[l, r]$.

Algorithm

Consider any 2 points m_1 , and m_2 in this interval: $l < m_1 < m_2 < r$. We evaluate the function at m_1 and m_2 , i.e. find the values of $f(m_1)$ and $f(m_2)$. Now, we get one of three options:

- $f(m_1) < f(m_2)$

The desired maximum can not be located on the left side of m_1 , i.e. on the interval $[l, m_1]$, since either both points m_1 and m_2 or just m_1 belong to the area where the function increases. In either case, this means that we have to search for the maximum in the segment $[m_1, r]$.

- $f(m_1) > f(m_2)$

This situation is symmetrical to the previous one: the maximum can not be located on the right side of m_2 , i.e. on the interval $[m_2, r]$, and the search space is reduced to the segment $[l, m_2]$.

- $f(m_1) = f(m_2)$

We can see that either both of these points belong to the area where the value of the function is maximized, or m_1 is in the area of increasing values and m_2 is in the area of descending values (here we used the strictness of function increasing/decreasing). Thus, the search space is reduced to $[m_1, m_2]$. To simplify the code, this case can be combined with any of the previous cases.

Thus, based on the comparison of the values in the two inner points, we can replace the current interval $[l, r]$ with a new, shorter interval $[l', r']$. Repeatedly applying the described procedure to the interval, we can get an arbitrarily short interval. Eventually, its length will be less than a certain pre-defined constant (accuracy), and the process can be stopped. This is a numerical method, so we can assume that after that the function reaches its maximum at all points of the last interval $[l, r]$. Without loss of generality, we can take $f(l)$ as the return value.

We didn't impose any restrictions on the choice of points m_1 and m_2 . This choice will define the convergence rate and the accuracy of the implementation. The most common way is to choose the points so that they divide the interval $[l, r]$ into three equal parts. Thus, we have

$$m_1 = l + \frac{(r - l)}{3}$$

$$m_2 = r - \frac{(r - l)}{3}$$

If m_1 and m_2 are chosen to be closer to each other, the convergence rate will increase slightly.

Run time analysis

$$T(n) = T(2n/3) + O(1) = \Theta(\log n)$$

It can be visualized as follows: every time after evaluating the function at points m_1 and m_2 , we are essentially ignoring about one third of the interval, either the left or right one. Thus the size of the search space is $2n/3$ of the original one.

Applying [Master's Theorem](#), we get the desired complexity estimate.

The case of the integer arguments

If $f(x)$ takes integer parameter, the interval $[l, r]$ becomes discrete. Since we did not impose any restrictions on the choice of points m_1 and m_2 , the correctness of the algorithm is not affected. m_1 and m_2 can still be chosen to divide $[l, r]$ into 3 approximately equal parts.

The difference occurs in the stopping criterion of the algorithm. Ternary search will have to stop when $(r - l) < 3$, because in that case we can no longer select m_1 and m_2 to be different from each other as well as from l and r , and this can cause an infinite loop. Once $(r - l) < 3$, the remaining pool of candidate points $(l, l + 1, \dots, r)$ needs to be checked to find the point which produces the maximum value $f(x)$.

Golden section search

In some cases computing $f(x)$ may be quite slow, but reducing the number of iterations is infeasible due to precision issues. Fortunately, it is possible to compute $f(x)$ only once at each iteration (except the first one).

To see how to do this, let's revisit the selection method for m_1 and m_2 . Suppose that we select m_1 and m_2 on $[l, r]$ in such a way that $\frac{r-l}{r-m_1} = \frac{r-l}{m_2-l} = \varphi$ where φ is some constant. In order to reduce the amount of computations, we want to select such φ that on the next iteration one of the new evaluation points m'_1, m'_2 will coincide with either m_1 or m_2 , so that we can reuse the already computed function value.

Now suppose that after the current iteration we set $l = m_1$. Then the point m'_1 will satisfy $\frac{r-m_1}{r-m'_1} = \varphi$. We want this point to coincide with m_2 , meaning that $\frac{r-m_1}{r-m_2} = \varphi$.

Multiplying both sides of $\frac{r-m_1}{r-m_2} = \varphi$ by $\frac{r-m_2}{r-l}$ we obtain $\frac{r-m_1}{r-l} = \varphi \frac{r-m_2}{r-l}$. Note that $\frac{r-m_1}{r-l} = \frac{1}{\varphi}$ and $\frac{r-m_2}{r-l} = \frac{r-l+m_2-m_1}{r-l} = 1 - \frac{1}{\varphi}$. Substituting that and multiplying by φ , we obtain the following equation:

$$\varphi^2 - \varphi - 1 = 0$$

This is a well-known golden section equation. Solving it yields $\frac{1 \pm \sqrt{5}}{2}$. Since φ must be positive, we obtain $\varphi = \frac{1 + \sqrt{5}}{2}$. By applying the same logic to the case when we set $r = m_2$ and want m'_2 to coincide with m_1 , we obtain the same value of φ as well. So, if we choose $m_1 = l + \frac{r-l}{1+\varphi}$ and $m_2 = r - \frac{r-l}{1+\varphi}$, on each iteration we can re-use one of the values $f(x)$ computed on the previous iteration.

Implementation

```
double ternary_search(double l, double r) {
    double eps = 1e-9;           //set the error limit here
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;
        double f1 = f(m1);       //evaluates the function at m1
        double f2 = f(m2);       //evaluates the function at m2
        if (f1 < f2)
            l = m1;
        else
            r = m2;
    }
    return f(l);                 //return the maximum of f(x) in [l, r]
}
```

Here `eps` is in fact the absolute error (not taking into account errors due to the inaccurate calculation of the function).

Instead of the criterion `r - l > eps`, we can select a constant number of iterations as a stopping criterion. The number of iterations should be chosen to ensure the required accuracy. Typically, in most programming challenges the error limit is 10^{-6} and thus 200 - 300 iterations are sufficient. Also, the number of iterations doesn't depend on the values of l and r , so the number of iterations corresponds to the required relative error.

Contributors:

likecs (39.32%) Yury Semenov (17.09%) Mariia Mykhailova (16.24%) Abhijeet Krishnan (11.11%)
 Oleksandr Kulkov (5.98%) Morass (5.13%) Joaquin Garmendia Cabrera (1.71%) Ahmed Marouf (0.85%)
 Lovish Madhu (0.85%) Puneet Kumar (0.85%) jxu (0.85%)

Original

Simulated Annealing

Simulated Annealing (SA) is a randomized algorithm, which approximates the global optimum of a function. It's called a randomized algorithm, because it employs a certain amount of randomness in its search and thus its output can vary for the same input.

The problem

We are given a function $E(s)$, which calculates the energy of the state s . We are tasked with finding the state s_{best} at which $E(s)$ is minimized. **SA** is suited for problems where the states are discrete and $E(s)$ has multiple local minima. We'll take the example of the [Travelling Salesman Problem \(TSP\)](#).

Travelling Salesman Problem (TSP)

You are given a set of nodes in 2 dimensional space. Each node is characterised by its x and y coordinates. Your task is to find an ordering of the nodes, which will minimise the distance to be travelled when visiting these nodes in that order.

Motivation

Annealing is a metallurgical process, wherein a material is heated up and allowed to cool, in order to allow the atoms inside to rearrange themselves in an arrangement with minimal internal energy, which in turn causes the material to have different properties. The state is the arrangement of atoms and the internal energy is the function being minimised. We can think of the original state of the atoms, as a local minima for its internal energy. To make the material rearrange its atoms, we need to motivate it to go across a region where its internal energy is not minimised in order to reach the global minima. This motivation is given by heating the material to a higher temperature.

Simulated annealing, literally, simulates this process. We start off with some random state (material) and set a high temperature (heat it up). Now, the algorithm is ready to accept states which have a higher energy than the current state, as it is motivated by the high temperature. This prevents the algorithm from getting stuck inside local minimas and move towards the global minima. As time progresses, the algorithm cools down and refuses the states with higher energy and moves into the closest minima it has found.

The energy function $E(s)$

$E(s)$ is the function which needs to be minimised (or maximised). It maps every state to a real number. In the case of TSP, $E(s)$ returns the distance of travelling one full circle in the order of nodes in the state.

State

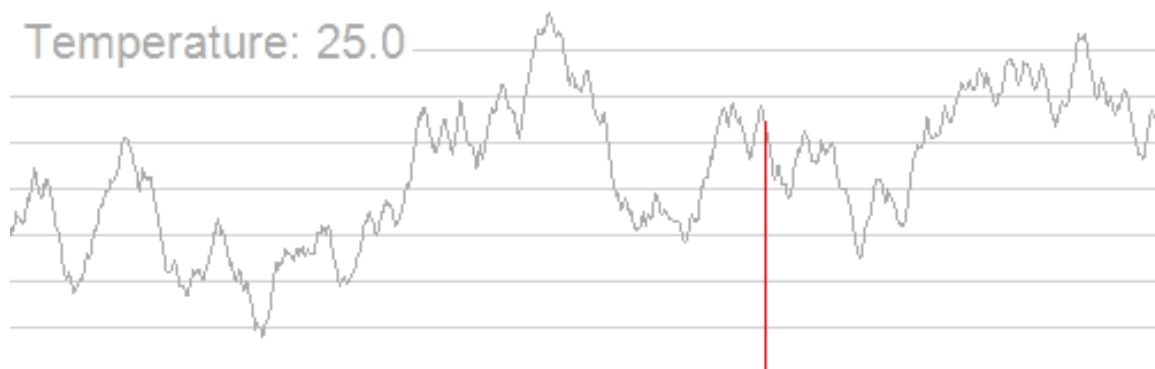
The state space is the domain of the energy function, $E(s)$, and a state is any element which belongs to the state space. In the case of TSP, all possible paths that we can take to visit all the nodes is the state space, and any single one of these paths can be considered as a state.

Neighbouring state

It is a state in the state space which is close to the previous state. This usually means that we can obtain the neighbouring state from the original state using a simple transform. In the case of the Travelling Salesman Problem, a neighbouring state is obtained by randomly choosing 2 nodes, and swapping their positions in the current state.

Algorithm

We start with a random state s . In every step, we choose a neighbouring state s_{next} of the current state s . If $E(s_{next}) < E(s)$, then we update $s = s_{next}$. Otherwise, we use a probability acceptance function $P(E(s), E(s_{next}), T)$ which decides whether we should move to s_{next} or stay at s . T here is the temperature, which is initially set to a high value and decays slowly with every step. The higher the temperature, the more likely it is to move to s_{next} . At the same time we also keep a track of the best state s_{best} across all iterations. Proceeding till convergence or time runs out.



A visual representation of simulated annealing, searching for the maxima of this function with multiple local maxima.

Temperature(T) and decay(u)

The temperature of the system quantifies the willingness of the algorithm to accept a state with a higher energy. The decay is a constant which quantifies the "cooling rate" of the algorithm. A slow cooling rate (larger u) is known to give better results.

Probability Acceptance Function(PAF)

$$P(E, E_{next}, T) = \begin{cases} \text{True} & \text{if } \mathcal{U}_{[0,1]} \leq \exp(-\frac{E_{next}-E}{T}) \\ \text{False} & \text{otherwise} \end{cases}$$

Here, $\mathcal{U}_{[0,1]}$ is a continuous uniform random value on $[0, 1]$. This function takes in the current state, the next state and the temperature, returning a boolean value, which tells our search whether it should move to s_{next} or stay at s . Note that for $E_{next} < E$, this function will always return True, otherwise it can still make the move with probability $\exp(-\frac{E_{next}-E}{T})$, which corresponds to the [Gibbs measure](#).

```
bool P(double E, double E_next, double T, mt19937 rng){
    double prob = exp(-(E_next-E)/T);
    if(prob > 1) return true;
    else{
        bernoulli_distribution d(prob);
        return d(rng);
    }
}
```

Code Template

```
class state {
public:
    state() {
        // Generate the initial state
    }
    state next() {
        state s_next;
        // Modify s_next to a random neighboring state
        return s_next;
    }
    double E() {
        // Implement the energy function here
    };
};

pair<double, state> simAnneal() {
    state s = state();
    state best = s;
    double T = 10000; // Initial temperature
    double u = 0.995; // decay rate
    double E = s.E();
    double E_next;
    double E_best = E;
    mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
    while (T > 1) {
        state next = s.next();
        E_next = next.E();
        if (P(E, E_next, T, rng)) {
            s = next;
            if (E_next < E_best) {

```

```

        best = s;
        E_best = E_next;
    }
    E = E_next;
}
T *= u;
}
return {E_best, best};
}

```

How to use:

Fill in the state class functions as appropriate. If you are trying to find a global maxima and not a minima, ensure that the $E()$ function returns negative of the function you are maximizing and print $-E_{best}$ in the end. Set the below parameters as per your need.

Parameters

- T : Initial temperature. Set it to a higher value if you want the search to run for a longer time.
- u : Decay. Decides the rate of cooling. A slower cooling rate (larger value of u) usually gives better results, at the cost of running for a longer time. Ensure $u < 1$.

The number of iterations the loop will run for is given by the expression

$$N = \lceil -\log_u T \rceil$$

Tips for choosing T and u : If there are many local minimas and a wide state space, set $u = 0.999$, for a slow cooling rate, which will allow the algorithm to explore more possibilities. On the other hand, if the state space is narrower, $u = 0.99$ should suffice. If you are not sure, play it safe by setting $u = 0.998$ or higher. Calculate the time complexity of a single iteration of the algorithm, and use this to approximate a value of N which will prevent TLE, then use the below formula to obtain T .

$$T = u^{-N}$$

Example implementation for TSP

```

class state {
public:
    vector<pair<int, int>> points;
    std::mt19937 mt{ static_cast<std::mt19937::result_type>(
        std::chrono::steady_clock::now().time_since_epoch().count()
    ) };
    state() {
        points = {{0,0},{2,2},{0,2},{2,0},{0,1},{1,2},{2,1},{1,0}} ;
    }
    state next() {
        state s_next;
        s_next.points = points;
        uniform_int_distribution<> choose(0, points.size()-1);
    }
}

```

```

        int a = choose(mt);
        int b = choose(mt);
        s_next.points[a].swap(s_next.points[b]);
        return s_next;
    }

    double euclidean(pair<int, int> a, pair<int, int> b) {
        return hypot(a.first - b.first, a.second - b.second);
    }

    double E() {
        double dist = 0;
        int n = points.size();
        for (int i = 0; i < n; i++)
            dist += euclidean(points[i], points[(i+1)%n]);
        return dist;
    };

    int main() {
        pair<double, state> res;
        res = simAnneal();
        double E_best = res.first;
        state best = res.second;
        cout << "Length of shortest path found : " << E_best << "\n";
        cout << "Order of points in shortest path : \n";
        for(auto x: best.points) {
            cout << x.first << " " << x.second << "\n";
        }
    }
}

```

Further modifications to the algorithm:

- Add a time based exit condition to the while loop to prevent TLE
- The decay implemented above is an exponential decay. You can always replace this with a decay function as per your needs.
- The Probability acceptance function given above, prefers accepting states which are lower in energy because of the $E_{next} - E$ factor in the numerator of the exponent. You can simply remove this factor, to make the PAF independent of the difference in energies.
- The effect of the difference in energies, $E_{next} - E$, on the PAF can be increased/decreased by increasing/decreasing the base of the exponent as shown below:

```

bool P(double E, double E_next, double T, mt19937 rng) {
    double e = 2; // set e to any real number greater than 1
    double prob = pow(e, -(E_next-E)/T);
    if (prob > 1)
        return true;
    else {
        bernoulli_distribution d(prob);
        return d(rng);
    }
}

```

